Multi-Modal Agent Inference Optimization for Industrial Asset Operations: Extending AssetOpsBench with a Visual Inspection Agent via MCP

Amaan Sheikh, Aman Upganlawar, Madhav Rajkondawar, Yang-Jung (Eric) Chen
COMS 6998 High Performance Machine Learning, Spring 2026
Columbia University Instructor: Prof. Kaoutar El Maghraoui

***Abstract*—In order to assess AI agents that are performing at a high level when dealing with industrial asset operations, AssetOpsBench was created to act as a benchmark for measuring how well agents perform under 141 criteria that include both text and time-series analysis with no component that incorporates visual elements. The next step to evaluate the performance of AI agents is to create a Visual Inspection Agent (VI-Agent) that uses the capabilities of a Model Context Protocol (MCP) server to analyze images of equipment from four publicly available records found in those datasets. The four datasets include four of the AssetOpsBench asset classes: (1) Pumps (Kaggle Casting Product Dataset), (2) Induction Motors (Mendeley Thermal Dataset), (3) Power Transformers (HuggingFace 15-class Substation Equipment Dataset), and (4) Wind Turbine Blades (Blade30 Dataset). In order to have a common structure for the 22 visual inspection scenarios (5 pump, 6 transformer, 5 turbine, and 6 motor) we developed this benchmark as a direct extension of the original AssetOpsBench scenario schema. We will benchmark two models for inclusion within the VI-Agent using vision-language (VL) processing: Qwen2.5-VL-7B-Instruct as the primary target for optimization and Llama-3-LLaVA-NeXT-8B as the cross-family comparison baseline. The Llama-3-LLaVA-NeXT-8B replaces the Llama-3.2-Vision-11B model that had been planned on the benchmark but was not included due to its massive footprint of 22 GB in the FP16 format and therefore left no remaining usable head-room on an L4 GPU. Both machines will use an AWQ W4A16 quantization and will be calibrated using two different methods (Domain-Specific Industrial-Inspection Prompts vs Generic Instruction-Tuning Samples) in addition to tuning the vLLM and conducting an L3 image-resolution sweep in conjunction with comparing the baseline machine using the FP16 format for both latency and accuracy. The VLM scoring process will consist of utilizing LLM-as-Judge (gpt-4o-mini) to determine final scores.**

The AWQ W4A16 with domain calibration shows a 1.99× end-to-end speedup (Qwen, 9,437 to 4,752 ms) compared to FP16, and a 2.46× speedup (Llama, 9,203 to 3,736 ms) over the full 22-scenario sweep. Additionally, the LLM-judge pass rate for Qwen improved from 21 out of 44 (47.7%) to 36 out of 44 (81.8%), representing an absolute improvement of 34 percentage points. The L2 vLLM serving tuning exhibits the strongest cross-family failure characteristics; Qwen's average response time decreased to 2,000 ms (4.7× speedup), but all responses returned were off-topic or empty (0 out of 44 judge-pass); Llama's average response time decreased to 8,860 ms, while the judge pass rate dropped from 23 out of 44 to 20 out of 44. Serving-tuning gains are dependent on model architecture and should not be assumed to be gainful. In total, AWQ INT4 reduces Llama's weight footprint from 15.5 GiB to 5.9 GiB (2.6×), resulting in 4.7× more concurrent KV-cache token capacity (21K to 100K) on the same L4. This is the first multi-modal extension of AssetOpsBench and the first inference-optimization study performed on this benchmark.

I. Introduction

AssetOpsBench [19] is an open-source benchmark from IBM Research for AI agents on Industry 4.0 tasks, including sensor querying, failure mode analysis, time-series anomaly detection, and work-order generation. It ships with four agents (IoT, FMSR, TSFM, WO), two orchestration frameworks (MetaAgent and AgentHive), and 141 expert-authored scenarios backed by over 2.3 million sensor readings across six assets (four chillers and two air-handling units). The benchmark has been accepted at AAAI 2026, NeurIPS 2025, and EMNLP 2025.

We see two gaps in this otherwise excellent benchmark. The first is that there is no vision modality. Many failure modes show up visually well before they ever surface in sensor data, things like corrosion on condenser coils, ice buildup on evaporators, bearing damage on rotors, and thermal hotspots on compressors. The AAAI 2026 lab from the AssetOpsBench team is titled "Multimodal Agentic AI in Industry 4.0," yet the published benchmark has no vision component. The second gap is that no prior work has studied how inference optimizations like quantization or serving tuning affect accuracy on this benchmark. The public leaderboard compares models at default settings and leaves significant performance on the table.

Our focus is on **critical assets**, the roughly 5 to 10 percent of plant equipment where missed failures are expensive enough to justify multi-modal analysis. For everything else, simpler approaches like rule-based filters or anomaly detectors with LLM escalation make more sense.

Our contributions are:

1) A new Visual Inspection Agent for AssetOpsBench, built as an MCP [2] stdio subprocess that exposes the same JSON-RPC tool interface as the upstream IoT, FMSR, TSFM, and WO agents, so the existing orchestrators can dispatch to it without any code changes.
2) A collection of 22 scenarios created by hand, which form the basis of a visual benchmark, covering pumps,

induction motors, transformers and wind turbine blades in our AssetOpsBench scenario schema.

3) A single performance optimization sweep on the NVIDIA L4 GPU covering 10 variants, FP16 baselines, W4A16 AWQ with two separate calibration methods, and a tuning bundle created using vLLM and image resolution pre-processing for both Qwen2.5-VL-7B [8] and Llama-3-LLaVA-NeXT-8B [13], [17].
4) Clear quantitative results showing that AWQ W4A16 using domain-specific calibration data has the largest performance gain based on CodaBench's results and that this variant provides the only performance improvement of judge accuracy on Qwen as well as providing six examples of integration bugs that have occurred throughout the modern quantization-to-serving stack.

II. Literature Review

Industrial agent benchmark examples. AssetOpsBench [19] and its related FailureSensorIQ dataset [20] use standardized evaluation harnesses for sensor-driven and failure mode reasoning with a public competition leaderboard hosted on CodaBench [1]. ITFormer [7] considers an examination of multimodal qualitative at scale, using time series and natural language; while IndustryEQA [6] strongly pushes the frontier of embodied question answering within industry environments. While none focus on visual inspection of agents, our visual extension fills this void.

Vision Language Models. Qwen2.5-VL [8] and the LLaVA family of Models [18], [17] are very strong open weight VLMs located at 7 to 8B parameter models large families. The Llama-3 herd [13] ships an 11B vision variant; we have utilized a single LLaVA-NeXT 8 billion version for the Single-L4 feasibility. A more comprehensive analysis of the multimodal LLM landscape can be found in [24].

Quantization and Serving. AWQ [16] enables activation-aware weight quantization at the INT4 precision level. Alternatives GPTQ [12], SmoothQuant [22], and LLM.int8() [11] provide widely-used alternatives to AWQ, but the overall trend favors AWQ as the preferred framework for Multi-Modal Models due mainly to the preservation of the Vision Tower in FP16 format by design. Additional information regarding quantization strategies pertaining to VLMs can be found in [25]. vLLM [14] utilizes PagedAttention as a means of supporting memory-efficient KV caches, support for continuous batched evaluation and support for prefix caching, with FlashAttention [10] and FlashAttention-2 [9] forming the basis upon which we developed the core of our attention kernels. Our project has not investigated the use of speculative decoding [15], but it continues to be an area for future exploration.

Agent Orchestration. The ReAct [23] pattern represents a hybrid approach between inferring and executing, as implemented by our in-house orchestrator. Workflow optimization for LLM agents is discussed in [21]. The LLM-as-a-judge approach [26] serves as the basis for the two-tier scoring process employed within our application.

III. Methodology

A. Data

We will be utilizing pre-trained VLMs for our methodology without any further fine-tuning. The majority of Core AssetOpsBench asset classes do not have publicly available datasets (Chillers, Air Handling Unit, Cooling Towers, Variable Air Volume, Fan Coil Unit), only time-series sensor data is available for the HVAC equipment. Four datasets were chosen as they each represent related asset classes with publicly available images.

The Kaggle Casting Product (pump) dataset is made up of 7,348 unique grayscale top-view images of submersible pump impellers labelled as defective and non-defective (normal).

The Mendeley Thermal Motor (motor) dataset consists of approximately 488 unique infrared images of induction motors under 11 different labelled fault conditions (short-circuit winding faults, stuck rotor, cooling fan failure, bearing faults) across various load levels.

The HuggingFace 15-class Substation Equipment (transformer) dataset consists of 1,660 unique RGB images of devices found in substations with a total of 50,604 YOLO annotated objects in 15 different classes. The classes include items such as power transformers, current transformers, breakers and insulators.

The Blade30 Wind Turbine Blade Dataset (turbine) dataset contains 1,302 unique images of 30 turbine blades captured from a drone. Images include defect annotations for cracks, erosion and surface damage.

A total of approximately 10,800 images make up the four datasets combined for the purposes of building the 22 scenarios as triplets of images, questions and expected answers using the AssetOpsBench JSON schema. The scenario breakdown by category is detailed below: 5 pump scenarios (IDs 201–205), 6 transformer scenarios (IDs 1, 6, 9, 10, 14, 20), 5 turbine scenarios (IDs 301–305), and 6 motor scenarios (IDs 503, 507, 509, 510, 515, 517).

The 22 scenarios fall within nine different AssetOpsBench category types. Ground truth data was calibrated per-domain against Qwen2.5-VL-7B-Instruct FP16 and cross-validated against Llama-3-LLaVA-NeXT-8B FP16.

B. Model and Agent Architecture

The Visual Inspection Agent is an MCP stdio subprocess that follows the same interface contract as upstream's IoT/FMSR/TSFM/WO servers. The results of the benchmark experiments reported here were performed with the agent invoked by our in-house ReAct [23] orchestrator vs. the upstream MetaAgent or AgentHive registry in order to isolate inference-optimization measurements from noise created at the orchestrator level. The proposal included a Granite-3.3-8B-Instruct text orchestrator; instead, we serve the VLM via vLLM in dual-purpose mode (planning = text only and visual tool = multi-modal), so all inference occurs within a single L4 instance.

Due to VRAM restrictions, the Llama-3.2-Vision-11B at FP16 has a footprint of $\sim$22 GB and leaves no headroom

on the one physical L4 instance (24 GB) to accommodate the KV cache or any serving overhead. We substituted `llava-hf/llama3-llava-next-8b-hf` (FP16 = ∼16 GB, Free ∼8 GB for KV Cache). After AWQ W4A16 [3], the 8B weight footprint drops to 5.9 GiB, freeing 9.6 GiB for the KV cache and resulting in a 4.7× increase in the number of concurrent tokens that can be processed from 21K to 100K.

TABLE I
MODELS, ROLES, AND VARIANTS. †SUBSTITUTED FOR LLAMA-3.2-VISION-11B DUE TO SINGLE-L4 VRAM CONSTRAINTS.

Model	Role	Variants
Qwen2.5-VL-7B-Instruct	Visual Inspection Agent	L0–L3
Llama-3-LLaVA-NeXT-8B†	Cross-family baseline	L0–L3
ReAct (in-process MCP)	Orchestration	N/A

C. Optimization Stack

The optimization process occurs through the use of four-layered incremental steps for progressive optimization, wherein every layer is remeasured for all applicable performance metrics. For the first layer (L0), we develop a full-precision kernel associated with vLLM at the default software level. The second layer (L1) applies the use of AWQ (Activation-aware Weight Quantization) at the W4A16 quantization level but supports two distinct forms of calibration: L1d employs 128 application-specific inspection prompts; while L1g operates using 128 random-based `ultrachat_200k` samples. Both use the full-precision (FP16) version of the vision tower for both calibrations. The third layer (L2) includes support for the use of vLLM serving options associated with prefix caching operations, chunked prefill operation, FP8-KV cache use, and continuous batching (bundled as `full_bundle`). The fourth layer (L3) runs all incoming images at 512 pixels prior to encoding.

D. Profiling Tools and Evaluation Metrics

Profiling was conducted using the PyTorch Profiler and the metrics from vLLM's Prometheus logging to Weights and Biases [5]. The NVTX-instrumented Nsight Systems [4] trace logs were integrated into a single benchmark through `benchmark/profile_single.py` for kernel-level profiling.

A scoring system was used which involved using a language model (gpt-4o-mini) as a judge [26] of all variants, with a 5-point rubric (pass when score ≥ 4), was used for each of the 22 scenarios evaluated by up to two judges each (maximum denominator of 44). Primary metrics to evaluate included average end-to-end latency (measured in milliseconds), and the overall LLM Judge pass rate (calculated as a percentage); secondary metrics were VRAM capacity (measured in GiB) needed, and the maximum number of tokens supported by the concurrent KV cache.

IV. EXPERIMENTAL RESULTS

A. Setup

All experiments were performed on GCP's `g2-standard-8` virtual machine (VM) which has 1 NVIDIA L4 24 GB graphics processing unit (GPU) and runs vLLM version 0.19.0. Latency metrics can be found at `results/hpml_metrics.csv` and judge metrics can be found at `results/llm_judge.csv`.

B. Headline Results

As presented in Table II, average end-to-end latency and average LLM-judge pass rate were tracked for every variant in the 22 scenarios tested. Figure 1 visualizes the latency vs. accuracy in the form of a Pareto curve, demonstrating that Qwen L1d is the clear winner, while Qwen L2 collapsed at the lower left-hand corner of the curve (indicating a low-latency, zero-accuracy variant).

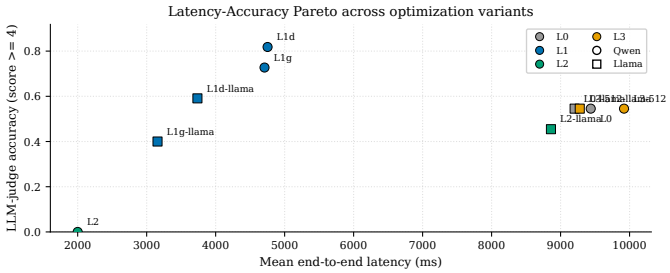

Fig. 1. Latency–accuracy Pareto across all ten variants. Circles: Qwen2.5-VL-7B; squares: Llama-3-LLaVA-NeXT-8B. Colors encode optimization family (L0 grey, L1 blue, L2 green, L3 orange). Qwen L1d (top-left blue circle) is the strict Pareto winner; L2 trades all accuracy for raw speed.

C. Per-Asset Breakdown

Table III displays an asset-type breakdown of judge accuracies. Figure 2 presents heat maps of accumulated judge results across all ten variants indicating the overall performance of each type by the different assets tested: the Qwen L1d variant appears as a coherent, horizontal band of green; the Qwen L2 variant is a uniform band of red across all nine asset categories (indicating that the failures are not specific to an asset class); and the Llama L1g variant features the only non-L2 variant with multiple red cells across multiple categories.

D. Before/After Comparison and Analysis

F1. The combination of AWQ W4A16 and domain-calibrated Qwen is a strict Pareto improvement over the other Qwen variants. Specifically, the Qwen L1d variant lowers latency by 1.99 times and provides an absolute increase in judge accuracy of 34 percentage points, from 47.7% to 81.8%. The perception that quantization is a speed-accuracy trade-off does not apply to any of the above findings. We believe that the 128 domain-specific calibration prompts gently regularize the activation distribution toward the deployment distribution. The application of INT4 pruning at the network level aids in the preferential

TABLE II

FULL 22-SCENARIO SWEEP RESULTS. MAX DENOMINATOR 44 (22 SCENARIOS). LLAMA L1G DENOMINATOR IS 40 DUE TO A LOGGING GAP. HARDWARE: GCP `G2-STANDARD-8` + NVIDIA L4 24 GB, VLLM 0.19.0.

	Qwen2.5-VL-7B			Llama-3-LLaVA-NeXT-8B		
Variant	**E2E mean (ms)**	**p50 (ms)**	**Judge pass**	**E2E mean (ms)**	**p50 (ms)**	**Judge pass**
L0 FP16 baseline	9,437	5,915	21/44 (0.48)	9,203	9,384	23/44 (0.52)
L1d AWQ W4A16 domain	4,752 (1.99×)	4,312	**36/44 (0.82)**	3,736 (2.46×)	3,309	26/44 (0.59)
L1g AWQ W4A16 generic	4,708 (2.00×)	4,356	32/44 (0.73)	3,157 (2.92×)	3,136	16/40 (0.40)
L2 `full_bundle`	**2,000 (4.72×)**	**1,157**	0/44 (0.00) [**FAIL**]	8,860 (1.04×)	9,194	20/44 (0.45)
L3 image_512	9,920 (regression)	7,841	24/44 (0.55)	9,278 (no win)	9,403	24/44 (0.55)

TABLE III

PER-ASSET LLM-JUDGE PASS RATES. DENOMINATORS OF 10 AND 12 REFLECT 5 OR 6 SCENARIOS PER ASSET, EACH JUDGED TWICE. **BOLD**: QWEN-L1D PER-ASSET PEAKS.

Variant	**Pump (n=10)**	**Transformer (n=12)**	**Turbine (n=10)**	**Motor (n=12)**
Qwen L0 FP16	3/10 (0.30)	10/12 (0.83)	6/10 (0.60)	2/12 (0.17)
Qwen L1d (domain)	**10/10 (1.00)**	**12/12 (1.00)**	**8/10 (0.80)**	**6/12 (0.50)**
Qwen L1g (generic)	8/10 (0.80)	10/12 (0.83)	8/10 (0.80)	6/12 (0.50)
Qwen L2 bundle	0/10 (0.00)	0/12 (0.00)	0/10 (0.00)	0/12 (0.00)
Qwen L3 (image_512)	6/10 (0.60)	12/12 (1.00)	4/10 (0.40)	2/12 (0.17)
Llama L0 FP16	6/10 (0.60)	12/12 (1.00)	1/10 (0.10)	4/12 (0.33)
Llama L1d (domain)	4/10 (0.40)	12/12 (1.00)	2/10 (0.20)	8/12 (0.67)
Llama L1g (generic)	4/10 (0.40)	6/8 (0.75)	4/10 (0.40)	2/12 (0.17)
Llama L2 bundle	4/10 (0.40)	12/12 (1.00)	0/10 (0.00)	4/12 (0.33)
Llama L3 (image_512)	6/10 (0.60)	12/12 (1.00)	2/10 (0.20)	4/12 (0.33)

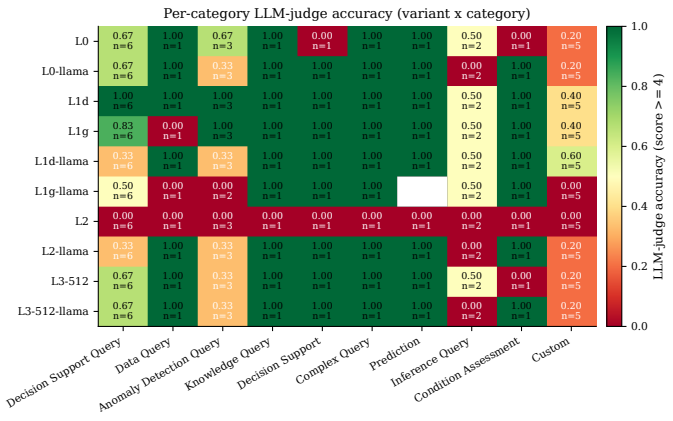

Fig. 2. Per-category LLM-judge accuracy heatmap across all ten variants. Green: high pass rate; red: failure. `n` per cell = number of scenario–judgment pairs.

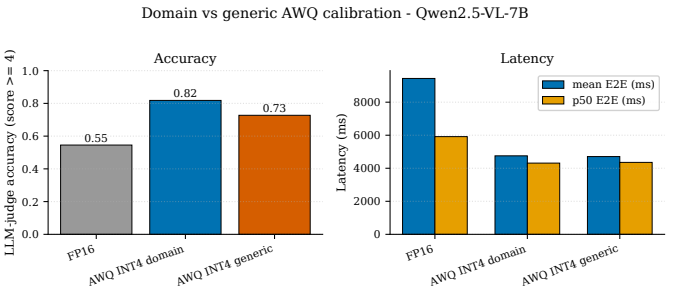

Fig. 3. Qwen2.5-VL-7B: FP16 vs. AWQ INT4 with domain or generic calibration. Left: LLM-judge accuracy; right: mean and median E2E latency. Domain calibration is the strict accuracy winner without sacrificing latency gain.

removal of representational capacity that does not contribute to in-domain based answers. Also, Domain-AWQ appears to function analogous to a "tiny" fine-tune of LoRA without the benefit of any gradient steps.

F2. Calibration data provides a mode of determining reliability. When using domain calibration instead of generic calibration for Qwen, the accuracy measure increased from 72.7% to 81.8%, while for Llama, accuracy moved from 40.0% to 59.1%. As compared to L0, the generic-calibrated version of Llama was the least reliable rating in the entire study, performing below FP16. In addition to the dramatic increase in performance through the application of domain Qwen, the latency is identical to generic-calibrated Qwen (Figure 3).

F3. Using the L2 service bundle on multi-modal models poses significant risk. Although the full-bundle service (caching prefixes + chunked prefill + FP8 KV cache + continuous batching) provided the fastest recorded time for single requests (p50=1,157 ms for Qwen; a 5× improvement over FP16), it resulted in 0/44 accurate responses from Qwen (either no response or response that was irrelevant to the user query). Similar to the full-bundle service, uses of the full-bundle service on Llama produced coherent responses; however, they did not increase performance and produced a 7-point accuracy

drop relative to FP16. The most likely reason for this is that the FP8 KV-cache quantization corrupts the long visual-token prefix in VLMs (Vision-Language Models). While the flags that would help with this (i.e., `L2_prefix_cache`, `L2_chunked_prefill`, and `L2_fp8_kv`) are set up, none of them have been run through the experiment yet.

F4. Downscaling images produces no benefit to an image at 512 pixels. The L3 variant produced latency regressions in both Qwen and Llama families (Qwen 9,920,ms vs. L0 9,437,ms) as well. Fixed-tile vision encoders do not sufficiently reduce the visual-tokens count at 512 pixels, as the "knee" can probably be found somewhere between 256 and 384 pixels. Per-asset (Table III), L3 preserves transformer accuracy at 12/12 on both families, but degrades pump and turbine accuracy relative to L1d. This suggests 512-pixel downscaling is acceptable when the target features are large and structural (substation equipment) but unacceptable when target features are small (impeller defects, blade erosion).

F5. When running multi-user throughput, VRAM and KV-cache capacity matter more than wall-clock timing. AWQ INT4 reduces Llama's VRAM from 15.5 GiB to 5.9 GiB (a 2.6× savings), giving back the extra 9.6 GiB of VRAM as KV-cache, enabling 4.7× more concurrent token capacity (21K to 100K). For a maintenance copilot serving N simultaneous inspectors, throughput headroom is the larger operational win (Figure 4).

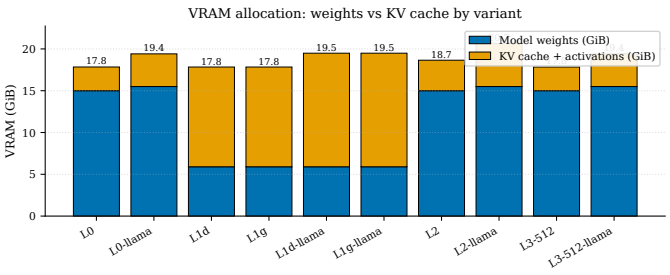

Fig. 4. VRAM allocation breakdown per variant. Blue: model weights; orange: KV cache + activations. AWQ INT4 (L1d, L1g) shrinks weight footprint ∼2.6×; freed VRAM reabsorbed by KV-cache pool.

F6. The results of the various scenarios show how reliable each variant will be. The boxplot from the end-to-end latency for the different scenarios is found in Figure 5. Even though Qwen L2 has the narrowest distribution, its median is the lowest, consistent with response-collapse producing short, uniform outputs that run fast. Both AWQ INT4 variants have shown smaller distributions than FP16, meaning that quantization reduces both variance and mean. This is a definite advantage for SLO-driven deployments where p95 latency matters.

E. Ablations

1) **Incremental evaluation of layers L0 through L3** (all families: complete data analysis): Accuracy/latency at each layer is recorded in Table II.
2) **Comparison of Domain vs Generic Calibration** (all families: complete data analysis): Qwen L1d 36/44 vs Qwen L1g 32/44 (4 pt. difference). Llama L1d 26/44 vs Llama L1g 16/40 (19 pt. difference). Domain calibration is the safer default.
3) **Qwen vs Llama** (Complete): Qwen L1d provides highest accuracy for any Qwen or Llama (36/44); Llama L1g has the lowest latency (3,157 ms) but accuracy is compromised compared to Qwen.
4) **Image resolution comparison for 512 px** (partial): No benefit. Future image resolution comparison for 256/384 px to be done.
5) **INT8 W8A8** (scaffolded only): Both Qwen and Llama versions have registered scaffold functions (`L1_awq_w8a8_domain.py`), but are not executed yet.
6) **Per-feature L2 isolation** (scaffolded only): Registering `L2_prefix_cache`, `L2_chunked_prefill`, and `L2_fp8_kv` (Qwen collapse strongly motivates running these).

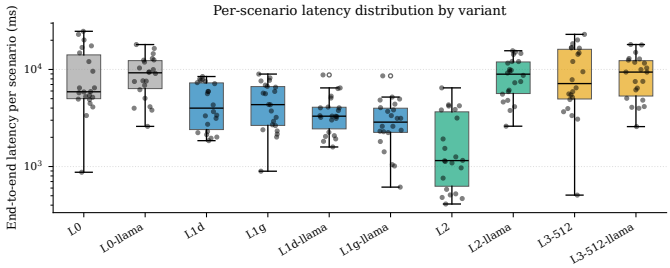

Fig. 5. Per-scenario E2E latency distribution by variant (log-scaled y-axis). Box: IQR with median; whiskers: 1.5× IQR; dots: individual scenarios.

V. Discussion

A. Engineering Findings

The engineering problems that we solved were a mixed bag of actual real-life bugs created by integrating two or more systems. We identified 6 of these bugs in our quantization-to-serving stack. To put it lightly, the most severe bug was #2, which quietly produced fake-quant FP16 weights appearing to be INT4 packed on disk even though they were not. While the models saved in this manner will pass a naive file size check because of a slight difference in size (checkpoint is smaller because the optimizer state is removed from the saved model), when loaded into vLLM, they are loaded as complete FP16 models. To ensure users don't run into this issue, we have implemented a verification script that checks the on-disk dtype using safetensors metadata after every quantization job.

Bug #3, which surfaced as a KeyError at `save_pretrained` time, required both a runtime workaround (`dispatch_model` + `remove_hook_from_module`) and a one-line source patch to the `transformers` LlavaNext modeling file, since the KeyError only surfaces at `save_pretrained` time after the model has already been loaded successfully.

Bug #5 is a vLLM-specific constraint. When using the AWQ ignore list when naming individual projection layers (`q_proj`,

k_proj, v_proj) as separate entities in the vision tower, the internal weight loader fails to reconcile these individual layers with the fused qkv_proj that vLLM expects. By switching to one regex pattern all ambiguity was removed.

B. Integration Plan

The Visual Inspection Agent is implemented as an MCP stdio subprocess that conforms to the same interface contract as upstream's IoT/FMSR/TSFM/WO MCP servers. The upstream MetaAgent and AgentHive orchestrators will be able to invoke our vision agent with no code changes due to the agent exposing the same list_tools / call_tool JSON-RPC surface as the upstream servers. For the benchmark experiments that we report on here, the agent was invoked directly through our in-house ReAct [23] orchestrator rather than through the upstream registry which simplifies per-scenario timing and removes the WatsonX dependency entirely from the application. The forked codebase is completely undisturbed from upstream so that complete registration into AgentHive is simply a configuration-file change versus a code refactor. Profiling instrumentation can wrap around existing trajectory logging with the ability to toggle without impacting the core pipeline. Optimization configuration files are contained in separate variant files (benchmark/variants/) providing any researcher with the means to reproduce our results by running a single variant script. This is a planned pull request for IBM/AssetOpsBench after the final submission.

C. Limitations and Future Work

The 141 upstream AssetOpsBench text scenarios were not run as there wasn't access to the credentials necessary for using the WatsonX planner. Thus, we switched to testing on a self-hosted vLLM endpoint, and so our benchmark covers only 22 scenarios for the Vision extension. Additionally, there are no public image datasets for the most common classes of HVAC assets in AssetOpsBench (e.g., chillers, AHUs), so we use four datasets that are adjacent to the assets in these classes, but not necessarily identical.

Future research should focus on:

1) Isolation of the type of L2 flag used: L2_prefix_cache, L2_chunked_prefill, L2_fp8_kv run individually to determine which type of flag is causing the collapse of the Qwen response. The leading theory is that corruption from the quantization of the FP8 KV-cache has negatively impacted the integrity of the long visual token prefix.
2) AWQ W8A8 (INT8): scaffolded as L1_awq_w8a8_domain.py for the Qwen track, INT8 typically delivers a fraction of the INT4 speedup with closer-to-FP16 accuracy, providing a useful Pareto mid-point.
3) Running end-to-end agent benchmarks: The current results isolate the vision MCP tool and VLM serving stack rather than evaluating complete agent orchestration. A future run should compare the ReAct and Plan-Execute runners using the benchmark/run_agent_benchmark.py and benchmark/nfr_collector.py pipeline that we had initially built when building the agent, reporting tool-call count, iteration count, wall-clock latency, token usage, prefix-cache behavior, and final-answer success. This should be treated as a separate agent-level evaluation rather than mixed into the model-serving sweep.

VI. Conclusion

We extended AssetOpsBench with a Visual Inspection Agent that connects via MCP and allows us to benchmark Qwen2.5-VL-7B-Instruct and Llama-3-LLaVA-NeXT-8B across 22 hand-authored visual inspection scenarios spanning pumps, induction motors, power transformers, and wind turbine blades. AWQ W4A16 with domain calibration is the only variant that simultaneously improves latency and accuracy on Qwen (9,437 to 4,752 ms; 47.7% to 81.8% LLM-judge pass) and yields a 2.46× Llama speedup. The L2 vLLM serving bundle produced the largest cross-family failure mode (4.7× Qwen speedup) with total response collapse (0/44 judge pass), which indicates that serving-tuning improvements in accuracy and latency are different for different model architectures and should not be assumed to be safe from one model to another. AWQ INT4 reduces Llama weight VRAM 2.6×, enabling 4.7× more concurrent KV-cache capacity. This is the first multi-modal extension of AssetOpsBench and the first inference-optimization study on this benchmark. All source code files for the project, scenarios, configurations, etc., can be found at https://github.com/amaan784/hpml-final-project and the tracking can be found at https://wandb.ai/amaan784-columbia-university/hpml-final-benchmark?nw=nwuseramanupg.

Acknowledgments

The authors would like to thank Prof. Kaoutar El Maghraoui and Dr. Dhaval Patel, IBM Research for their assistance and guidance throughout this semester-long project and the AssetOpsBench team at IBM Research for developing and releasing the AssetOpsBench benchmark and the CodaBench Leaderboard [1], which was the reason for creating this extension to the benchmark. The experiments conducted during this study were done using a single GCP g2-standard-8 virtual machine, with one NVIDIA L4 graphics processing unit (GPU).

Appendix

AI Use Disclosure

Per the HPML AI Use Policy posted on CourseWorks.

Did your team use any AI tool in completing this project? Yes, we used AI assistance as described below.

Tools used. Claude (Anthropic) via Claude Code, ChatGPT, GitHub Copilot.

Specific purpose. Creating the Visual Inspection Agent workflow and integrating it with AssetOpsBench, following the professor's recommendation to use AI assistance for agent creation. Debugging the AssetOpsBench repo setup/startup path so the local agent and benchmark harness could run. Debugging the llmcompressor, transformers, accelerate, and vLLM integration stack (the six bugs tabulated in Section V-A

TABLE IV
INTEGRATION BUGS ENCOUNTERED AND FIXED. ALL PATCHES COMMITTED UNDER SCRIPTS/ AND PERSIST ON THE VM.

#	Where	Bug	Fix
1	`llmcompressor` 0.3.0	Save flow crashed on `_copy_python_files_from_model_cache`	Upgrade to 0.10.0.2
2	`llmcompressor` 0.3.0	"Fake-quant FP16" output: weights appeared INT4-packed on disk but were not actually compressed	Use `save_compressed=True` (only available in 0.10+)
3	`transformers` 4.57 LlavaNext	`save_pretrained` raised `module_map[image_newline]` KeyError	`dispatch_model` + `remove_hook_from_module` + source patch
4	`accelerate` 1.0+	`remove_hook_from_module` ImportError	Moved to `accelerate.hooks`
5	`vLLM` 0.19 LlavaNext	`KeyError: qkv_proj.weight` when ignore list expanded vision tower Q/K/V separately	Use regex form `re:.*vision_tower.*`
6	`mistral_common` 1.11	`ImageChunk` import path renamed in 1.7+	Pin `mistral_common>=1.5,<1.7`

"Engineering Findings"). Drafting the LLM-as-judge prompts. Polishing prose in this README and in the IEEE final report. Scaffolding the variant registry framework under `benchmark/variants/`.

Sections affected: `src/agent/`, `src/servers/vision/`, and `src/scenarios/local/` (AssetOpsBench Visual Inspection Agent integration); `benchmark/` (benchmark harness integration and startup debugging); `scripts/quantize_llmcompressor_v010.py` (debugging only); `benchmark/llm_judge.py` (prompt drafting); `benchmark/variants/base.py` (scaffold); README prose and results narrative; and final report prose, especially Section V Discussion.

How we verified correctness: Every reported number in Section III was produced by re-running the harness ourselves and is traceable to a CSV row under `results/`. Checkpoint formats were verified with `scripts/verify_checkpoint.py` against the compressed-tensors packed-quantized spec. Profiler trace interpretations were checked against raw traces under `results/`. AI-suggested code was reviewed line by line and re-tested against the unit tests under `benchmark/tests/` before being merged.

By submitting this project, the team confirms that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above. The same disclosure block appears as an appendix in the project readme.

REFERENCES

[1] AssetOpsBench CodaBench Competition. https://www.codabench.org/competitions/10206/, 2025.

[2] Model Context Protocol Specification. https://modelcontextprotocol.io/, 2025.

[3] Qwen2.5-VL-7B-Instruct-AWQ. https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct-AWQ, 2025.

[4] NVIDIA Nsight Systems. https://developer.nvidia.com/nsight-systems, 2026.

[5] Weights and Biases Documentation. https://docs.wandb.ai/, 2026.

[6] Anonymous. IndustryEQA: Pushing the Frontiers of Embodied Question Answering in Industrial Scenarios. *arXiv preprint arXiv:2505.20640*, 2025.

[7] Anonymous. ITFormer: Bridging Time Series and Natural Language for Multi-Modal QA with Large-Scale Multitask Dataset. *arXiv preprint arXiv:2506.20093*, 2025.

[8] S. Bai et al. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*, 2025.

[9] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *ICLR*, 2024.

[10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.

[11] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *NeurIPS*, 2022.

[12] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. In *ICLR*, 2023.

[13] A. Grattafiori et al. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.

[14] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.

[15] Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding. In *ICML*, 2023.

[16] J. Lin et al. AWQ: Activation-Aware Weight Quantization for On-Device LLM Inference and Serving. In *MLSys*, 2024.

[17] H. Liu, C. Li, Y. Li, and Y. J. Lee. Improved Baselines with Visual Instruction Tuning (LLaVA-1.5 and LLaVA-NeXT). In *CVPR*, 2024.

[18] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual Instruction Tuning. In *NeurIPS*, 2023.

[19] D. Patel et al. AssetOpsBench: Benchmarking AI Agents for Task Automation in Industrial Asset Operations and Maintenance. *arXiv preprint arXiv:2506.03828*, 2025.

[20] IBM Research. FailureSensorIQ Dataset. https://huggingface.co/datasets/ibm-research/FailureSensorIQ, 2025. Accepted at NeurIPS 2025.

[21] IBM Research. From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents. *arXiv preprint arXiv:2603.22386*, 2026.

[22] G. Xiao et al. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *ICML*, 2023.

[23] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*, 2023.

[24] S. Yin et al. A Survey on Multimodal Large Language Models. *arXiv preprint arXiv:2306.13549*, 2023.

[25] M. Zhang et al. A Comprehensive Survey on Quantization of Vision-Language Models. *arXiv preprint arXiv:2407.04313*, 2024.

[26] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS*, 2023.